Scalable Web Application with ALB and Auto Scaling

Baseline Implementation Report (as built, before any modification)

Item	Details
Project	Project 1: Scalable Web Application with ALB and Auto Scaling (EC2-based architecture)
Tooling	Terraform (>= 1.5) with the HashiCorp AWS provider (~> 5.60)
Region	us-east-1 (two Availability Zones)
Report date	September 28, 2026
Purpose	Document the deployed baseline so that upcoming improvements can be compared against it

1. Executive summary

This report documents the first working version of Project 1, a production-style web platform on AWS built entirely with Terraform. The stack was deployed successfully in us-east-1. It serves a web page through CloudFront and an Application Load Balancer from an Auto Scaling group of EC2 instances that run in private subnets across two Availability Zones, backed by a Multi-AZ RDS MySQL database.

All eight service areas requested in the project brief are implemented in code: VPC, EC2 with Auto Scaling, ALB with WAF, CloudFront, RDS Multi-AZ, Route 53, Systems Manager, and CloudWatch with SNS. Route 53 is implemented but optional, and was not enabled in this deployment because no domain name was supplied.

Key results

- Deployment completed and returned all Terraform outputs (ALB DNS name, CloudFront domain, RDS endpoint, ASG name).
- The site loads through CloudFront and shows the serving instance ID and Availability Zone (verified by screenshot).
- The ALB layer 7 rule for /admin/* returns HTTP 403 (verified by captured output).
- Two obstacles were resolved during deployment: orphaned resources from an earlier apply, and AWS Free Tier restrictions on RDS.
- Several hardening and process gaps remain; they are listed in Section 9 and turned into a roadmap in Section 10.

2. Requirements coverage

The table maps each service listed in the project brief to what was implemented and where it lives in the code.

Requirement	Implementation	File
VPC: public and private subnets, NAT Gateway, Security Groups, NACLs	One VPC with 2 AZs and 3 tiers (public, private app, private DB). One NAT Gateway per AZ. Separate route tables. Custom NACLs for public and private subnets. Three chained Security Groups.	vpc.tf, security.tf
EC2 + ASG: Launch Template, target tracking	Launch Template on Amazon Linux 2023 (IMDSv2 required, encrypted gp3 volume). ASG min 2 / desired 2 / max 6 with ELB health checks and rolling instance refresh. Target tracking on CPU (50%) and on ALB requests per target (500). One step scaling policy for scale-out.	compute.tf
ALB + WAF: layer 7 routing, OWASP Top 10 rules	Internet-facing ALB in both public subnets, target group with /health check, listener rule that returns 403 for /admin and /admin/*. WAFv2 web ACL with AWS managed Common, KnownBadInputs, SQLi and Linux rule sets plus a rate limit of 2,000 requests per 5 minutes per IP.	alb.tf, waf.tf
CloudFront: cache static assets	Distribution with the ALB as origin. /static/* uses the managed CachingOptimized policy. All other paths use CachingDisabled. Viewers are redirected to HTTPS. Default CloudFront certificate. Price class 100.	cloudfront.tf
RDS Multi-AZ: MySQL/PostgreSQL with automated failover	MySQL 8.0, Multi-AZ, encrypted gp3 storage (20 GB, auto-scaling to 100 GB), placed in isolated DB subnets. Master password generated and stored in Secrets Manager (not in Terraform state).	rds.tf
Route 53: alias record and health checks	Alias A record to the ALB and an HTTP(S) health check on /health. Optional: created only when domain_name and hosted_zone_id are set. Not enabled in this deployment.	route53.tf
Systems Manager: Session Manager	EC2 IAM role with AmazonSSMManagedInstanceCore and CloudWatchAgentServerPolicy. No key pair and no port 22 anywhere.	iam.tf

Requirement	Implementation	File
CloudWatch + SNS: dashboards, alarms, notifications	SNS topic with an email subscription. Five alarms (ASG CPU, ALB 5xx, unhealthy hosts, RDS CPU, RDS free storage). One dashboard with four widgets.	monitoring.tf

3. Architecture

Traffic path: Internet users reach CloudFront over HTTPS. CloudFront forwards to the ALB through the Internet Gateway, and the WAF web ACL inspects every request at the ALB. The ALB distributes traffic to EC2 instances in the private app subnets. The instances talk to RDS on port 3306 in the isolated DB subnets. Outbound internet access for the instances (package installs) goes through the NAT Gateway in their own AZ. Administrators reach instances only through Session Manager.

A full-page architecture diagram follows. Note that the CIDR labels printed on the subnets in that diagram are off by one. The correct values are in the addressing table below and in the Terraform code.

Network addressing (from the Terraform code)

Tier	AZ A	AZ B	Notes
VPC	10.0.0.0/16		DNS support and hostnames enabled
Public subnets	10.0.0.0/24	10.0.1.0/24	ALB, NAT Gateways. Route to Internet Gateway.
Private app subnets	10.0.10.0/24	10.0.11.0/24	EC2 (ASG). Default route to the NAT Gateway of the same AZ.
Private DB subnets	10.0.20.0/24	10.0.21.0/24	RDS only. No route to the internet.

Security controls

Control	Rule
ALB Security Group	Inbound 80 and 443 from 0.0.0.0/0. All outbound.
Web Security Group	Inbound 80 from the ALB Security Group only. All outbound.
RDS Security Group	Inbound 3306 from the Web Security Group only.
Public NACL	Inbound 80, 443, ephemeral ports 1024-65535, and all traffic from the VPC CIDR. All outbound.
Private NACL (app and DB)	Inbound all traffic from the VPC CIDR and ephemeral ports 1024-65535 (return traffic through NAT). All outbound.
Instance hardening	IMDSv2 required, encrypted EBS, no public IP, no key pair, no SSH.
Data protection	RDS storage encrypted. Master password managed by RDS in Secrets Manager. DB subnets have no internet route.

Architecture diagram (as built)

Scalable Web Application on AWS (EC2 + ALB + Auto Scaling + RDS Multi-AZ)

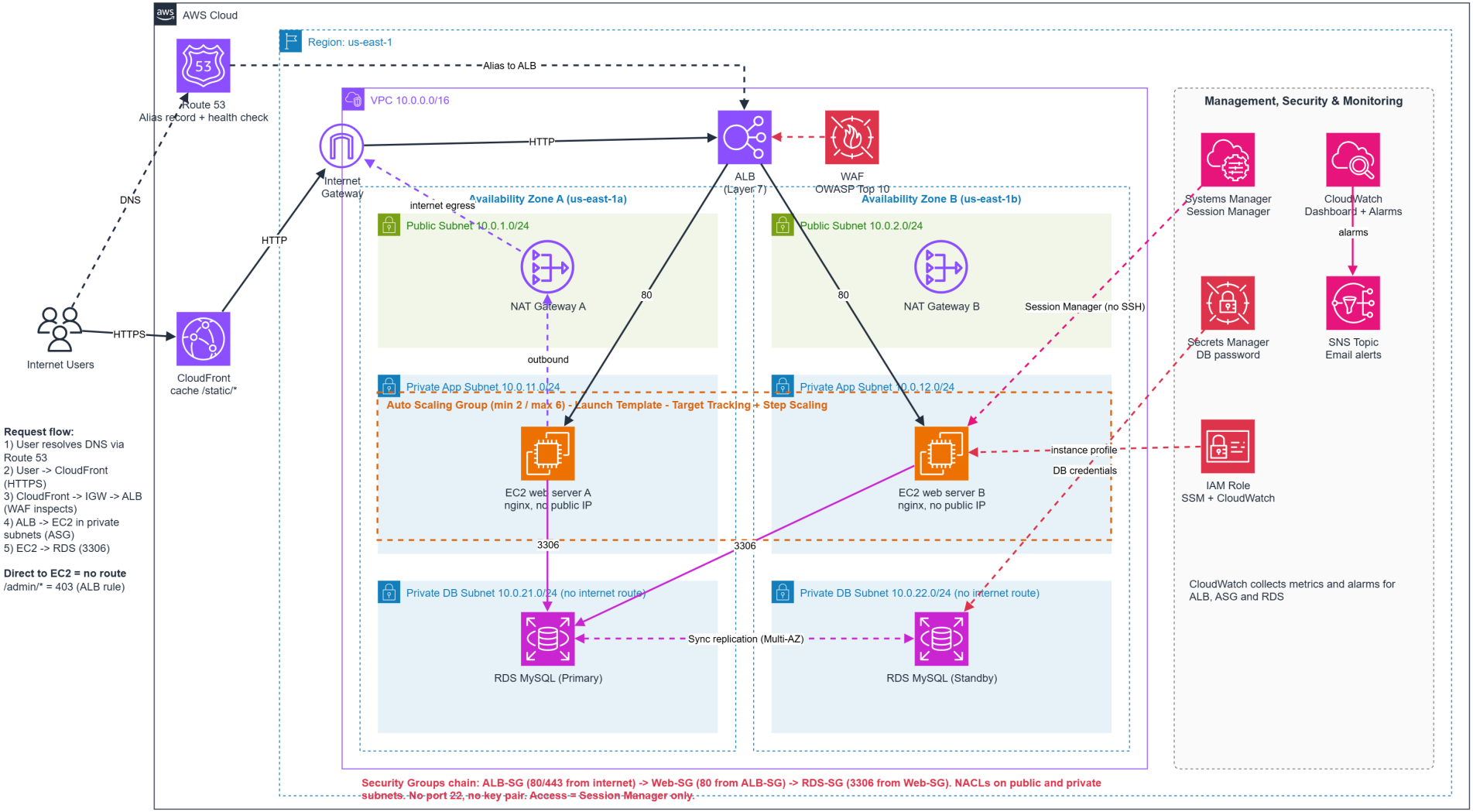


Figure 1. Draw.io diagram of the deployed architecture. Known cosmetic issues are listed in Section 9.

4. Terraform project structure

The project is a single flat root module. Each file owns one concern.

File	Responsibility
versions.tf	Terraform and provider version constraints, provider configuration, default tags (Project, Environment, ManagedBy).
variables.tf	Input variables: region, CIDR, instance type, ASG sizes, DB settings, alert email, optional domain and certificate.
vpc.tf	VPC, subnets, Internet Gateway, NAT Gateways and Elastic IPs, route tables, NACLs.
security.tf	ALB, web and RDS Security Groups.
iam.tf	EC2 role, policy attachments, instance profile.
user_data.sh	Bootstraps nginx, a demo page showing instance ID and AZ, /static/style.css and the /health endpoint.
compute.tf	AMI lookup, Launch Template, Auto Scaling group, three scaling policies.
alb.tf	ALB, target group, HTTP listener (HTTPS listener when a certificate is given), /admin rule.
waf.tf	WAFv2 web ACL and ALB association.
rds.tf	DB subnet group and Multi-AZ MySQL instance.
cloudfront.tf	CloudFront distribution and cache behaviors.
route53.tf	Optional alias record and health check.
monitoring.tf	SNS topic and subscription, CloudWatch alarms and dashboard.
outputs.tf	ALB DNS, CloudFront domain and URL, RDS endpoint and secret ARN, ASG name, Session Manager hint.

Key variable defaults

Variable	Default	Variable	Default
region	us-east-1	asg_min / desired / max	2 / 2 / 6
instance_type	t3.micro	db_instance_class	db.t3.micro
single_nat_gateway	false (one NAT per AZ)	db_skip_final_snapshot	true
vpc_cidr	10.0.0.0/16	certificate_arn / domain_name	empty (HTTP only, no DNS)

5. Deployment result

The final apply completed and produced the following outputs (account-specific identifiers omitted).

Output	Value
app_url_via_cloudfront	https://d2g9xj66sc72pj.cloudfront.net
alb_dns_name	scalable-web-dev-alb-1691664699.us-east-1.elb.amazonaws.com
rds_endpoint	scalable-web-dev-mysql.c6jmgmaauz41.us-east-1.rds.amazonaws.com:3306
asg_name	scalable-web-dev-asg

Approximate resource inventory: 1 VPC, 6 subnets, 1 Internet Gateway, 2 NAT Gateways with Elastic IPs, 4 route tables, 2 NACLs, 3 Security Groups, 1 IAM role and instance profile, 1 Launch Template, 1 Auto Scaling group with 3 scaling policies, 1 ALB with target group, listener and rule, 1 WAF web ACL, 1 Multi-AZ RDS instance with subnet group, 1 CloudFront distribution, 1 SNS topic with subscription, 5 CloudWatch alarms and

1 dashboard.

6. Testing and validation

The table separates results that were confirmed with evidence in this project from results that the owner reported but did not attach.

Test	Expected	Status	Evidence
Site through CloudFront	Page shows instance ID and AZ	**Verified**	Screenshot: instance i-0b6d2cc971111a07c, AZ us-east-1b
ALB layer 7 rule	/admin/ returns 403	**Verified**	curl output: HTTP/1.1 403 Forbidden, Server: awselb/2.0, body 'Forbidden'
WAF blocks script injection	HTTP 403 from WAF	**Reported by owner**	Output not shared
CloudFront caching of /static/*	Miss, then Hit	**Reported by owner**	Output not shared
Session Manager access	Shell without SSH	**Reported by owner**	Output not shared
Auto Scaling under load	Instance count rises above 2	**Reported by owner**	Output not shared
RDS forced failover	AZ changes, service recovers	**Reported by owner**	Output not shared

For a formal hand-in, capture the missing outputs and screenshots and attach them here.

7. Issues encountered and how they were resolved

#	Symptom	Cause	Resolution
1	Apply failed with 'already exists' for the ALB, ASG, DB subnet group and WAF web ACL.	An earlier apply had created the same named resources, but the current Terraform state did not track them. Two VPCs tagged Project=scalable-web existed.	Searched local state files to confirm no other project owned these names. Deleted the orphaned ASG, ALB (after disassociating the WAF), DB subnet group and WAF web ACL, then the orphaned VPC with its NAT Gateways, Elastic IPs, Security Groups, subnets, route tables, NACLs and Internet Gateway. Re-ran apply successfully.
2	RDS creation failed with FreeTierRestrictionError.	The account is on a Free Tier plan that limits the backup retention period.	Set backup_retention_period to 1 and disabled Performance Insights in rds.tf. Apply completed. Multi-AZ was kept and worked.
3	Emoji in the demo page displayed as garbled characters.	The generated HTML had no character set declaration.	Added a UTF-8 meta tag to user_data.sh. Applying it triggers a rolling instance refresh. The fix was delivered; confirm it is applied.

Lesson: the orphaned-resource problem was caused by losing track of Terraform state. Remote state with locking (Section 10) removes this class of failure.

8. Cost awareness

The following components are billed by the hour while they exist, even when idle. Destroy the environment after testing with `terraform destroy`.

- Two NAT Gateways (set `single_nat_gateway = true` to run one during labs).
- Application Load Balancer.

- RDS Multi-AZ instance and its storage.
- EC2 instances in the Auto Scaling group.
- CloudFront, WAF and CloudWatch are usage-based and small at lab traffic levels.

9. Known limitations of the baseline

Security and architecture

- The ALB accepts traffic from the whole internet, so users can bypass CloudFront. This was visible when the ALB DNS name was called directly.
- CloudFront talks to the ALB over plain HTTP. HTTPS is coded but not enabled because no certificate or domain was supplied.
- The WAF protects the ALB only, not CloudFront itself.
- RDS uses AWS-managed keys, has deletion protection off, skips the final snapshot, and keeps only 1 day of backups (Free Tier limit).
- No CloudTrail, VPC Flow Logs, ALB access logs or WAF logging. No VPC endpoints (Session Manager and S3 traffic goes through NAT).
- The instances run a static nginx page, not a real application, so the database connection is provisioned but not exercised. The CloudWatch agent is not installed, so memory and disk metrics are missing.
- Step scaling is scale-out only. Scale-in relies on the target tracking policies.

Process and code quality

- Terraform state is local, with no locking and no versioning. This directly caused issue 1.
- The code is a flat root module with a single environment. No modules, no dev/prod separation, no input validation.
- No CI checks (fmt, validate, tflint, security scanning) and no cost estimation on changes.

Documentation

- The diagram subnet CIDR labels are off by one (see Section 3 for the correct values).
- The diagram implies Route 53 fronts the application, but in this deployment users enter through the CloudFront URL and Route 53 is not enabled.
- Cosmetic overlaps in the rendered diagram: Route 53 label over its icon, ALB and WAF labels overlapping the AZ titles, the Session Manager label crossed by a line, the ASG title touching the subnet titles, and the Security Group note crossing the region border.

10. Recommended next steps

Ordered by value and effort. Items 1 to 3 give the biggest gain for the least work.

Priority	Improvement	Why it matters
1	Remote state in S3 with versioning, encryption and locking	Prevents lost state and orphaned resources; standard practice for team work.
2	Restrict the ALB to CloudFront (managed prefix list plus a secret origin header, default action 403)	Closes the CloudFront bypass so the WAF and cache cannot be skipped.
3	Enable HTTPS end to end with ACM and Route 53, redirect HTTP to HTTPS	Encrypts traffic between CloudFront and the ALB; code is already prepared.

Priority	Improvement	Why it matters
4	Refactor into modules (vpc, compute, database, edge, monitoring) with dev and prod environments	Reusability, clearer ownership, safer changes.
5	VPC endpoints (SSM, S3), KMS customer-managed keys, Flow Logs, CloudTrail, ALB and WAF logs	Stronger security posture and full audit trail.
6	Production settings for RDS: deletion protection, final snapshot, 7-day backups, Performance Insights (after leaving Free Tier)	Data safety.
7	CI/CD with GitHub Actions: fmt, validate, tflint, tfsec or Checkov, plan on pull request, gated apply, Infracost	Repeatable, reviewed and cost-aware changes.
8	Replace the demo page with a small real app using Secrets Manager; add CloudWatch agent; bake an AMI with Packer	Proves the three tiers work end to end and speeds up instance start.
9	Fix diagram CIDR labels and overlaps; add a legend	Accurate documentation for hand-in.